

Ex7.

```
/* *****  
 * ex789-prod-con-threads.cpp  
 * Producer-consumer synchronisation problem in C++  
 * ***** */  
  
#include <stdio>  
#include <stdlib>  
#include <pthread.h>  
#include <signal>  
  
constexpr int PRODUCERS = 2;  
constexpr int CONSUMERS = 1;  
constexpr int BUFFER_SIZE = 10;  
  
int producer_buffer[BUFFER_SIZE];  
int consumer_sum = 0;  
int buffer_head = 0;  
int buffer_tail = 0;  
int buffer_count = 0;  
  
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;  
pthread_cond_t buffer_not_empty = PTHREAD_COND_INITIALIZER;  
pthread_cond_t buffer_not_full = PTHREAD_COND_INITIALIZER;  
  
void *producer(void *threadid) {  
    // Write producer code here  
    while (true) {  
        pthread_mutex_lock (&lock);  
        while (buffer_count == BUFFER_SIZE) {  
            pthread_cond_wait(&buffer_not_full, &lock);  
        }  
  
        producer_buffer[buffer_tail] = int(rand()) % 10 + 1;  
        printf("Producer: Generated %d\n", producer_buffer[buffer_tail]);  
        buffer_tail++;  
        buffer_tail %= BUFFER_SIZE;  
        buffer_count++;  
  
        pthread_cond_signal(&buffer_not_empty);  
        pthread_mutex_unlock (&lock);  
  
        usleep(150000);  
    }  
}
```

```

    pthread_exit(NULL);
}

void *consumer(void *threadid) {
    // Write consumer code here
    while (true) {
        pthread_mutex_lock (&lock);
        while (buffer_count == 0) {
            pthread_cond_wait(&buffer_not_empty, &lock);
        }

        consumer_sum += producer_buffer[buffer_head];
        buffer_head++;
        buffer_head %= BUFFER_SIZE;
        buffer_count--;
        printf("Consumer: Sum is now %d\n", consumer_sum);

        pthread_cond_signal(&buffer_not_full);
        pthread_mutex_unlock (&lock);

        usleep(150000);
    }
    pthread_exit(NULL);
}

void signal_handler(int signum) {
    printf("Interrupt signal (%d) received. Exiting...\n", signum);
    printf("Consumer sum is %d\n", consumer_sum);
    exit(signum);
}

int main(int argc, char *argv[]) {
    signal(SIGINT, signal_handler);

    pthread_t producer_threads[PRODUCERS];
    pthread_t consumer_threads[CONSUMERS];
    int producer_threadid[PRODUCERS];
    int consumer_threadid[CONSUMERS];

    // initialize
    consumer_sum = 0;

    int rc;
    int t1, t2;
    for (t1 = 0; t1 < PRODUCERS; t1++) {

```

```

        int tid = t1;
        producer_threadid[tid] = tid;
        printf("Main: creating producer %d\n", tid);
        rc = pthread_create(&producer_threads[tid], NULL, producer,
                           (void *)&producer_threadid[tid]);

        if (rc) {
            printf("Error: Return code from pthread_create() is %d\n", rc);
            exit(-1);
        }
    }

    for (t2 = 0; t2 < CONSUMERS; t2++) {
        int tid = t2;
        consumer_threadid[tid] = tid;
        printf("Main: creating consumer %d\n", tid);
        rc = pthread_create(&consumer_threads[tid], NULL, consumer,
                           (void *)&consumer_threadid[tid]);

        if (rc) {
            printf("Error: Return code from pthread_create() is %d\n", rc);
            exit(-1);
        }
    }

    pthread_exit(NULL);
}

```

Ex8.

```

/*****
 * ex789-prod-con-threads.cpp
 * Producer-consumer synchronisation problem in C++
 *****/

#include <time.h>
#include <cerrno>
#include <cstdio>
#include <cstdlib>
#include <iostream>
#include <fcntl.h>
#include <unistd.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <sys/types.h>

```

```

#include <semaphore.h>

constexpr int PRODUCERS = 2;
constexpr int CONSUMERS = 1;
constexpr int BUFFER_SIZE = 10;
constexpr int LIMIT = 20;

struct SharedMemory {
    int producer_buffer[BUFFER_SIZE];
    int consumer_sum;
    int buffer_head;
    int buffer_tail;
    int buffer_count;
    int limit;
};

void *producer(SharedMemory *shared, sem_t *lock, sem_t *buffer_not_full,
sem_t *buffer_not_empty) {
    // Write producer code here
    srand(getpid());
    while (shared->limit > 0) {
        shared->limit--;
        if(shared->buffer_count == BUFFER_SIZE) {
            sem_wait(buffer_not_full);
        }
        sem_wait(lock);

        shared->producer_buffer[shared->buffer_tail] = int(rand()) % 10 + 1;
        printf("Producer %d generating %dth number: Generated %d\n",
getpid(), LIMIT - shared->limit, shared->producer_buffer[shared-
>buffer_tail]);
        shared->buffer_tail++;
        shared->buffer_tail %= BUFFER_SIZE;
        shared->buffer_count++;

        sem_post(lock);
        sem_post(buffer_not_empty);

        usleep(150000);
    }
    exit(0);
}

```

```

void *consumer(SharedMemory *shared, sem_t *lock, sem_t *buffer_not_full,
sem_t *buffer_not_empty) {
    // Write consumer code here
    srand(getpid());
    int limit = LIMIT;
    while (limit-- > 0) {
        if(shared->buffer_count == 0) {
            sem_wait(buffer_not_empty);
        }
        sem_wait(lock);

        shared->consumer_sum += shared->producer_buffer[shared->buffer_head];
        shared->buffer_head++;
        shared->buffer_head %= BUFFER_SIZE;
        shared->buffer_count--;
        printf("Consumer: Sum is now %d\n", shared->consumer_sum);

        sem_post(lock);
        sem_post(buffer_not_full);

        usleep(150000);
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    // allocate shared memory
    srand((unsigned) time(NULL));
    int shmrand = rand();
    key_t shmkey = ftok("/dev/null", shmrand);
    printf("shmkey for shared memory = %d\n", shmkey);

    int shmid = shmget(shmkey, sizeof(SharedMemory), 0644 | IPC_CREAT);
    if (shmid < 0) {
        perror("shmget\n");
        exit(1);
    }

    // attach p to shared memory
    SharedMemory *shared = reinterpret_cast<SharedMemory *>(shmat(shmid,
NULL, 0));
    shared->consumer_sum = 0;
    shared->buffer_head = 0;
    shared->buffer_tail = 0;
    shared->buffer_count = 0;

```

```

shared->limit = LIMIT;
printf("Shared memory is allocated.\n\n");

// allocate semaphores
std::string lock_name = "semLock-" + std::to_string(shmrand);
sem_t *lock = sem_open(lock_name.c_str(), O_CREAT | O_EXCL, 0644, 1);
std::string bne_name = "semBufferNotEmpty-" + std::to_string(shmrand);
sem_t *buffer_not_empty = sem_open(bne_name.c_str(), O_CREAT | O_EXCL,
0644, 0);
std::string bnf_name = "semBufferNotFull-" + std::to_string(shmrand);
sem_t *buffer_not_full = sem_open(bnf_name.c_str(), O_CREAT | O_EXCL,
0644, 1);

    if ( fork () == 0) producer(shared, lock, buffer_not_full,
buffer_not_empty); // producer 1
    if ( fork () == 0) producer(shared, lock, buffer_not_empty,
buffer_not_empty); // producer 2
    consumer(shared, lock, buffer_not_empty, buffer_not_empty);

// Wait for children to finish
for (int i = 0; i < PRODUCERS; ++i) {
    wait(NULL);
}

// cleanup shared memory
// shared memory detach
shmdt(shared);
shmctl(shmid, IPC_RMID, 0);

// cleanup semaphores
sem_unlink(lock_name.c_str());
sem_close(lock);
sem_unlink(bne_name.c_str());
sem_close(buffer_not_empty);
sem_unlink(bnf_name.c_str());
sem_close(buffer_not_full);
exit(0);
}

```

Ex9.

Here are the output from running time ./labexe for both version when limit = 20:

Threads	
Consumer: Sum is now	
real	0m3.015s
user	0m0.004s
sys	0m0.007s

Process	
Consumer: Sum is now	
real	0m3.021s
user	0m0.012s
sys	0m0.005s

Here are the output from running time ./labexe for both version when limit = 100:

Threads	
Consumer: Sum is now	
real	0m15.028s
user	0m0.010s
sys	0m0.014s

Process	
Consumer: Sum is now	
real	0m15.029s
user	0m0.018s
sys	0m0.008s

The user and real time for threads version is faster than the process version. This might because the process version is more expensive for the CPU to handle shared memory and create new processes. However, as the limit increases, the difference between the real time for threads and processes decreases. This suggests that the process version might allow for better I/O usage since they have separate memory space and the OS might be more efficient at handling processes then threads when scheduling I/O tasks.